

Assignment 1 Report – Tejal Nair

1. Overview

This report documents the process of building a large-scale Python corpus, training a custom tokenizer, conducting the pre-training and fine-tuning stages of a language model for code understanding and masked-if prediction. The pipeline follows the assignment requirement of (i) constructing a sufficiently large dataset, (ii) training a tokenizer from scratch, (iii) pre-training a model on the resulting code corpus, and (iv) later fine-tuning and evaluating it on the (masked_function → if_header) task.

2. Dataset Construction

2.1 Source Repositories

A diverse set of public GitHub repositories was cloned from a curated list acquired using <https://seart-ghs.si.usi.ch/>. Each repository contained multiple Python projects of varying domains. After cloning roughly 5,812 repositories, we scanned for .py files, yielding approximately 194,000 candidate Python files.

2.2 Quality Filtering

To ensure data relevance and usability:

- Files were parsed with Python's ast module to verify syntactic validity.
- Files with at least one definable function and at least one *maskable* if statement inside functions were kept.
- Optional heuristics filtered out extremely short or auto-generated files and duplicate contents via SHA-1 hashes.

This stage reduced the corpus to ~20,000 clean Python files, each containing usable control-flow structures. The cleaning step ran in parallel using a process pool for speed.

2.3 Counting Maskable if Instances

To quantify usable examples, we counted all if statements inside function scopes across the cleaned corpus using AST traversal. Result: 252,805 maskable if statements, which is within the recommended number of 150,000 instances

for pre-training, as according to assignment instructions.

2.4 Building Function–Label Instances

Each function was then processed to create training examples where a single if header was masked with the special token [IF_MASK].

For every such mask:

- Input:** function text with one if header replaced by [IF_MASK] (indentation preserved).
- Label:** the exact original if ...: header line. Instances were deduplicated using SHA-1 content hashes and deterministically split into 80 / 10 / 10 train/validation/test sets.

Split	Instances
Train	176,226
Val	21,123
Test	21,262

This produced 218,611 unique function–header pairs as JSONL files (e.g. train.jsonl, val.jsonl, test.jsonl).

3. Tokenizer Training

3.1 Tokenizer Design

A Byte-Level BPE tokenizer was trained from scratch on the cleaned Python corpus using the tokenizers library. Vocabulary size was set to 32,000, with an initial alphabet from the byte-level pre-tokenizer to preserve all ASCII and indentation characters.

3.2 Special Tokens

The tokenizer included: [PAD], [UNK], [CLS], [SEP], [MASK], [IF_MASK]. [IF_MASK] serves as the task-specific placeholder for masked if statements. After training, it was exported as a Hugging Face compatible PreTrainedTokenizerFast and saved under artifacts/hf_tokenizer_fast/.

4. Pre-training Procedure

4.1 Objective

The goal of pre-training is to teach a model general code understanding through denoising auto-encoding: so given code with random tokens masked, the model reconstructs the original sequence.

4.2 Data Preparation

The cleaned corpus (data/instance_corpus/) contains over 20,000 Python files. During pre-training, these files are read and segmented into chunks of roughly 4,000 characters to produce manageable sequence lengths. Each chunk is tokenized with the custom Byte-Level BPE tokenizer (vocabulary ~ 32,000) and truncated to a maximum of 512 tokens.

Roughly 200,000 segments are used for training and 10,000 for validation.

Fifteen percent of non-special tokens in each sequence are randomly masked. The input to the model is this corrupted sequence. The label is the original unmasked sequence.

This denoising setup effectively teaches the model to “fill in” missing code, a task that later transfers to predicting masked if statements.

4.3 Model Parameters

A compact encoder–decoder configuration (*T5-tiny*) was constructed with the following parameters:

Parameter	Value
Layers	8
Hidden dimension	256
Feed-forward dimension	1024
Attention head	8
Dropout rate	0.10
Vocab size	32000
Initialization	Random with no pretrained weights

All embeddings, positional encodings, and attention weights are learned from scratch using the tokenizer’s vocabulary. This design makes the model small enough to train pretty quickly on local hardware while still capturing the necessary code structure.

4.4 Training Configuration

Training is performed with the Hugging Face Transformers library and PyTorch backend.

Key hyperparameters:

Setting	Value
Learning rate	3×10^{-4}
Optimizer	AdamW (fused)
Scheduler	Cosine decay + 3 % warm-up
Epochs	2
Batch size	$4 \times$ gradient accumulation $4 \rightarrow 16$
Max sequence length	512

Mask fraction	15%
Weight decay	0.01

4.5 Training Output

The model checkpoint and tokenizer are saved under runs/pretrain_denoise/final/. This checkpoint represents the fully pretrained *T5-tiny* model and will later serve as the initialization point for fine-tuning on the masked-if prediction task.

5. Fine-Tuning Procedure

5.1 Objective

After pre-training, the model was fine-tuned to specialize in predicting the masked if header within a Python function.

Each training instance consisted of a code fragment where one if statement was replaced by the special token [IF_MASK]; the expected output was the original if header (for example, if len(arr) > 0:).

This stage adapts the model’s generic code-completion ability to a narrower conditional-reasoning task.

5.2 Dataset and Sampling

The fine-tuning corpus was built from the 218,000 pre-processed instances generated during corpus construction. For efficiency, a mini subset of ~20,000 examples was sampled uniformly from the training pool, ensuring diversity across repositories and file types. Inputs were truncated to 128 tokens and labels to 32 tokens to stay within memory limits. A small validation split (~2,000 examples) was retained for quick monitoring, while the large-scale test and provided datasets were reserved for final evaluation.

5.3 Training Configuration

Fine-tuning was performed with the Hugging Face Transformers Trainer on top of the previously saved pre-training checkpoint.

Parameter	Value / Notes
Initialization	runs/pretrain_denoise_dev/final
Architecture	Same T5-tiny (8 layers, 256 hidden)
Learning rate	3×10^{-4} with cosine decay
Optimizer	Adafactor
Batch size	2 (per device) $\times$ accumulation 2 $\rightarrow$ effective 4
Epochs / Steps	~2000 gradient steps (~ 1 epoch over mini set)

Mask token	[IF_MASK] added to tokenizer vocabulary
Generation during training	Disabled for speed
Length bucketing	Enabled (group_by_length=True) to reduce padding overhead

To keep the process lightweight on a local macOS machine, data-loader multiprocessing was disabled, checkpointing and frequent evaluation were skipped, and only a final model snapshot was written to runs/finetune_ifmask_mini/final.

5.4 Rationale for Mini Fine-Tuning

Because full fine-tuning on $\approx 176,000$ instances exceeded available RAM, a parameter-efficient, small-subset training was used. This approach trades some final accuracy for quick iteration and ensures that learned representations from pre-training are retained while adapting to the new task.

6. Evaluation and Results

The evaluation stage assessed the performance of the fine-tuned model on two distinct test sets: the internally constructed generated test set and an externally provided test set.

For the generated test set, the evaluation dataset consisted of approximately 5,000 Python functions containing one masked conditional statement each, held out from the pretraining and fine-tuning corpora. To ensure fair testing, any examples in which the [IF_MASK] token had been truncated during tokenization were automatically filtered out. The evaluation employed greedy decoding with a maximum of 32 generated tokens and used canonicalized string comparison to mitigate the impact of formatting discrepancies such as inconsistent indentation, trailing colons, or differences in quotation style. Accuracy was computed as the exact-match ratio between the canonicalized predicted and expected strings.

Out of 5,000 examples, 4,279 were retained after filtering, while 721 were excluded due to truncation of the masked token. The resulting exact-match accuracy was 0.0%, indicating that the model was unable to correctly reconstruct any complete if headers in the held-out test set. (Results found in data/instances_built/provided_test.csv). This result suggests that the model failed to generalize meaningfully beyond the training data.

Several factors likely contributed to this outcome. First, the fine-tuning process was performed on a highly reduced training subset of approximately 20,000 examples (about one-tenth of the available data) to fit within limited memory constraints. As a result, the model's exposure to the diversity of Python conditional constructs was limited. Second, both the input and output sequence lengths during training were capped at 128 and 32 tokens, respectively, which may have truncated essential context needed to reconstruct the masked condition accurately.

Despite the poor numerical performance, the experiment demonstrates a complete end-to-end pipeline for code representation learning: repository-level data collection, corpus cleaning and filtering, tokenizer training, unsupervised pretraining, task-specific fine-tuning, and evaluation. The outcome highlights the importance of data scale and pretraining duration when training from scratch on programming languages.

The model was also evaluated on the provided test set. Since this dataset did not include explicitly masked conditionals, a preprocessing stage was implemented to construct evaluation instances. Each Python function in the dataset was parsed using the ast module to identify conditional (if) statements, and for each function, one such statement was masked and paired with its original header as the ground-truth label. This allowed us to generate (input, label) pairs consistent with the fine-tuning task format.

From this dataset, 294 instances were retained after filtering, with no examples dropped. Using the same decoding and canonical comparison procedures as before, the model again achieved 0% exact-match accuracy. (Results found in data/instances_built/provided_testset.csv). This confirms that the current fine-tuned checkpoint lacks sufficient representational capacity and task-specific adaptation to reconstruct missing if headers correctly.

This outcome, while numerically poor, provides valuable diagnostic evidence about the limitations of the current configuration. Because the model was trained on a minimal subset of the corpus and a lightweight architecture, its parameters are unlikely to have captured complex conditional patterns that depend on multi-line context or semantic reasoning over code variables.